

ECDSA 摘要未绑定时的存在性伪造 及 libsecp256k1 安全与性能分析

小组成员：宋佳原、宋曰琦、杜昊霖

2026 年 7 月

目录

1	实验目的	3
2	实验环境与材料	3
3	ECDSA 数学基础	3
3.1	secp256k1 参数	3
3.2	签名	4
3.3	验证	4
4	摘要未绑定时的存在性伪造	4
4.1	攻击构造	4
4.2	为什么这不是破解 ECDSA	5
5	实验实现	5
5.1	曲线运算	5
5.2	核心伪造代码	5
6	实验结果	6
7	libsecp256k1 接口审计	6
7.1	API 已明确警告该误用	6
7.2	Low-S 与本实验的区别	7
8	安全修复案例：编译器破坏常数时间性质	7
8.1	Clang 14 修复	7
8.2	GCC 13.1 ECDH 修复	7
8.3	数学上为什么需要常数时间	8

目录	2
9 性能改进及数学依据	8
9.1 ECDSA 验证的联合标量乘	8
9.2 GLV 自同态	8
9.3 生成元预计算与发布性能数据	9
9.4 safegcd 模逆	9
10 Bug 修复与性能改进汇总	9
11 对 Bitcoin 使用密码算法的启示	10
12 实验限制	10
13 结论	11
A 复现命令	11
B 参考资料	11

1 实验目的

本实验研究 ECDSA 验证接口直接接受 32 字节消息摘要时的安全边界。目标是在不知道私钥的条件下，按照课件给出的构造生成一个能够通过验证的摘要-签名对 (e', r', s') ，并证明该结果并不能伪造攻击者预先指定的消息。实验进一步检查 Bitcoin Core 官方 libsecp256k1 仓库，分析接口如何警告调用者绑定原始消息，并从密码算法角度研究常数时间修复、标量乘法优化、模逆算法和预计算表之间的安全-性能权衡。

2 实验环境与材料

项目	内容
实验平台	Ubuntu 20.04 / Windows WSL
实现语言	Python 3, 仅标准库
椭圆曲线	secp256k1
哈希函数	SHA-256
审计仓库	https://github.com/bitcoin-core/secp256k1
仓库提交	d2d04864ef9b056151603a3ced7980958b058028
报告编译	Windows TeX Live 2024, XeLaTeX

表 1: 实验环境

实验代码没有调用现成 ECDSA 库，而是实现有限域模逆、椭圆曲线点加、倍点和双倍加标量乘，以便直接观察数学等式。该实现仅用于教学，不具备生产环境需要的常数时间、故障攻击防护和密钥清理能力。

3 ECDSA 数学基础

3.1 secp256k1 参数

secp256k1 定义在有限域 $\mathbb{F}_p$ 上：

$$E : y^2 = x^3 + 7 \pmod{p},$$

其中

$$p = 2^{256} - 2^{32} - 977,$$

生成元为 G ，其素数阶为

$$n = \text{FFFEBAAEDCE6AF48A03BBFD25E8CD0364141}.$$

私钥 $d \in \mathbb{Z}_n^*$ ，公钥为 $P = dG$ 。

3.2 签名

令消息摘要整数为 $e = H(m) \bmod n$ ，签名者随机选择 $k \in \mathbb{Z}_n^*$ ，计算

$$R = kG = (x_R, y_R), \quad r = x_R \bmod n,$$

以及

$$s = k^{-1}(e + dr) \bmod n.$$

签名为 (r, s) 。如果 $r = 0$ 或 $s = 0$ ，应重新选择 k 。

3.3 验证

验证者首先检查 $1 \leq r, s < n$ ，然后计算

$$w = s^{-1} \bmod n, \quad u_1 = ew \bmod n, \quad u_2 = rw \bmod n,$$

$$R' = u_1G + u_2P = (x', y').$$

当 R' 不是无穷远点且 $x' \bmod n = r$ 时接受。正确性来自

$$\begin{aligned} R' &= es^{-1}G + rs^{-1}P \\ &= s^{-1}(e + rd)G \\ &= k(e + rd)^{-1}(e + rd)G = kG = R. \end{aligned}$$

4 摘要未绑定时的存在性伪造

4.1 攻击构造

攻击者只知道公钥 P ，任取非零 $u, v \in \mathbb{Z}_n^*$ ，计算

$$R' = uG + vP = (x', y'), \quad r' = x' \bmod n.$$

若 $r' = 0$ 则重新选择 u, v 。定义

$$s' = r'v^{-1} \bmod n, \quad qqquad e' = r'uv^{-1} \bmod n.$$

于是

$$(s')^{-1}r' = v \pmod{n}, \quad qqquad (s')^{-1}e' = u \pmod{n},$$

验证器计算得到

$$(s')^{-1}(e'G + r'P) = uG + vP = R'.$$

其横坐标模 n 恰为 r' ，所以 (r', s') 是摘要 e' 下的有效签名，而攻击者从未知道 d 。

4.2 为什么这不是破解 ECDSA

攻击者不能先指定消息 m^* 再要求 $e' = H(m^*)$ 。构造中的 e' 是由 u, v, r' 反推出来的随机标量。要把它解释成某个指定消息，需要找到

$$H(m^*) \equiv e' \pmod{n},$$

这相当于攻击 SHA-256 的原像性质。因此，本实验展示的是 existential forgery for an attacker-chosen digest，而不是在标准 EUF-CMA 模型下对指定消息的伪造。

真正的问题出在应用层：若验证方接收来自不可信输入的 `msghash32`，却没有自己从协议规定的消息序列化计算哈希，那么“签名验证通过”只证明某个椭圆曲线等式成立，不再证明签名者认可了业务消息。

5 实验实现

5.1 曲线运算

代码使用仿射坐标。不同点相加的斜率为

$$\lambda = (y_2 - y_1)(x_2 - x_1)^{-1} \pmod{p},$$

倍点斜率为

$$\lambda = 3x_1^2(2y_1)^{-1} \pmod{p}.$$

结果坐标为

$$x_3 = \lambda^2 - x_1 - x_2, y_3 = \lambda(x_1 - x_3) - y_1 \pmod{p}.$$

标量乘使用 double-and-add。所有参数固定为可复现实验值；固定 nonce 绝不能用于真实签名，因为重复或可预测 nonce 会泄露私钥。

5.2 核心伪造代码

```
def forge_for_chosen_digest(public_key, u, v):
    r_point = point_add(scalar_mul(u, G),
                        scalar_mul(v, public_key))
    r = r_point.x % N
    v_inv = inverse(v, N)
    s = r * v_inv % N
    e = r * u * v_inv % N
    return e, (r, s), r_point
```

程序同时执行四项检查：正常签名通过；伪造验证方程成立；伪造摘要通过；同一伪造签名对预先指定消息摘要失败。

6 实验结果

运行命令：

```
python3 ecdsa_digest_forgery.py | tee forgery_result.txt
```

实际输出如下：

```
=== secp256k1 ECDSA digest-forgery experiment ===
public key x      = 84054503a22cd9cb...aa86bdfecf8100d8
public key y      = 9e5c3164bea46138...f42d26b19389ee4c
SHA256(message)   = b4295ee1a2bf3ee9...c9f06aaa2ae934b4
legitimate r      = b268f9412d100717...edf3f4f42b70dedd
legitimate s      = d55b4d97cbceb1d7...2b0506d67ce8b8c0
legitimate verify  = PASS

attacker knows private key? NO
chosen u          = bb0c501e6b81f603...d311d711983c61ab
chosen v          = 4177418fd0530300...4d96046472632a61
R' = uG + vP, x'  = e165cd2be6927522...829064f926e1d440
forged digest e'  = 81767c7b1712a2b1...09f8480d64dd1bd0
forged r'         = e165cd2be6927522...829064f926e1d440
forged s'         = d95fec4a6c050882...62e434c00239c19c
verification equation R'=uG+vP = PASS
verify chosen digest e'          = PASS
e' == SHA256(target message)?    = False
verify target message hash       = FAIL (expected)

CONCLUSION:
PASS: a valid (digest, signature) pair was created without the private key.
PASS: it does NOT sign the chosen target message; ECDSA itself is not broken.
```

结果汇总见表 2。

实验同时给出了正例和反例：如果只展示“伪造摘要验证 PASS”，容易误解成 ECDSA 已被破解；目标消息验证 FAIL 则明确限定了结论。

7 libsecp256k1 接口审计

7.1 API 已明确警告该误用

当前官方仓库 include/secp256k1.h 中，secp256k1_ecdsa_verify 接收 32 字节 msghash32。其注释明确要求：验证者必须自行对消息应用密码哈希函数，不能直接接

检查项目	结果
使用私钥生成的正常签名验证	PASS
伪造点满足 $R' = uG + vP$	PASS
(e', r', s') 在公钥 P 下验证	PASS
e' 等于目标消息 SHA-256	False
(r', s') 验证目标消息摘要	FAIL (预期)

表 2: ECDSA 摘要伪造实验结果

受外部提供的摘要，否则无需私钥也很容易构造“有效”签名。这与本实验的数学构造完全对应。

这种 API 设计有现实原因：Bitcoin 的签名摘要并非简单的 $\text{SHA256}(m)$ ，而是由交易版本、输入、输出、金额、Script、locktime 和 sighash 类型按共识规则序列化后计算。底层曲线库不可能自行理解所有上层协议，因此摘要绑定必须由 Bitcoin Core 的交易签名逻辑完成。

7.2 Low-S 与本实验的区别

同一头文件还指出 ECDSA 存在 $(r, s) \mapsto (r, n - s)$ 的签名可塑性，库只接受 low-S 形式。Low-S 解决的是“同一消息具有两个等价签名”的规范化问题；本实验研究的是“应用接受攻击者选择的摘要”。前者不能修复后者，二者必须区分。

8 安全修复案例：编译器破坏常数时间性质

8.1 Clang 14 修复

官方 changelog 的 v0.3.1 记录：Clang 14 及更新版本可能将内存对象的条件移动编译为依赖秘密数据的控制流或内存访问，使库面临时间侧信道风险。修复目标是恢复 secret-independent control flow 和 secret-independent memory access，并加入 Wycheproof 的 Bitcoin low-S ECDSA 测试向量。

密码代码“源代码看起来无分支”并不充分。优化编译器可以识别掩码或条件移动语义，重新生成分支；若分支方向或查表地址依赖私钥位，攻击者可通过多次计时、cache 或分支预测观测逐步推断秘密。因而常数时间性质必须对最终机器码和具体工具链测试。

8.2 GCC 13.1 ECDH 修复

v0.3.2 又记录 GCC 13.1 对 ECDH 路径产生 secret-dependent control flow 的问题。这说明侧信道安全不是一次性属性：源代码、编译器版本、优化选项和 CPU 共同决定

实际执行轨迹。仓库后来增加未发布 GCC/Clang 快照测试，以便提前发现类似误编译。

8.3 数学上为什么需要常数时间

标量乘

$$Q = dP$$

若使用朴素 double-and-add，则每个私钥比特为 1 时多执行一次点加。运行时间近似与 Hamming weight $\text{wt}(d)$ 相关；查表算法若直接以秘密窗口为数组下标，还会泄露窗口值。libsecp256k1 对秘密标量使用无数据依赖分支、无秘密索引的条件移动，并提供 context randomization 作为纵深防御。

9 性能改进及数学依据

9.1 ECDSA 验证的联合标量乘

验证的主要成本是

$$R = u_1G + u_2P.$$

仓库 README 说明验证路径结合以下技术：

- wNAF (windowed non-adjacent form) 降低非零数字密度；
- 对固定生成元 G 使用更大窗口和预计算倍点；
- 使用 Shamir's trick 同时计算两个标量乘；
- 使用 secp256k1 的高效自同态将一个 256 位标量拆成两个约 128 位标量。

wNAF 表示保证非零数字之间至少有 $w - 1$ 个零，平均非零密度约为 $1/(w + 1)$ ，因此点加次数明显少于普通二进制展开。窗口越大，在线点加越少，但预计算表越大，形成时间-空间权衡。

9.2 GLV 自同态

secp256k1 存在易计算自同态

$$\phi(x, y) = (\beta x, y), \quad \phi(P) = \lambda P,$$

其中 $\beta^3 = 1 \pmod{p}$ 、 $\lambda^3 = 1 \pmod{n}$ 。库将

$$k = k_1 + \lambda k_2 \pmod{n}$$

分解为 $|k_1|, |k_2|$ 约 128 位，于是

$$kP = k_1P + k_2\phi(P).$$

两个较短标量可联合计算，减少加倍链长度。仓库 `src/scalar_impl.h` 还给出格基、舍入常量以及分解界的验证说明。

9.3 生成元预计算与发布性能数据

v0.5.0 更换签名和公钥生成使用的点乘算法，并把预计算表配置改为 2、22 或 86 KiB；v0.5.1 将默认表从 22 KiB 提升到 86 KiB。更大的固定基表用内存换取更少在线点运算。

v0.4.1 移除旧的手写 x86_64 域运算汇编，因为现代编译器生成了更高效代码。官方在 GCC 10.5.0 下报告，当原配置启用手写汇编时，修改后 `secp256k1_ecdsa_verify` 和 Schnorr 验证约加速 10%。这说明“汇编必然比 C 快”并不成立，持续基准测试比实现语言标签更可靠。

v0.2.0 则为 MSVC x86_64/arm64 增加 128 位宽乘法支持，官方报告约 20% 加速。底层原因是 4 个 64 位 limb 的标量/域运算可以直接利用宽乘积，减少拆分为较小 limb 时的指令和进位处理。

9.4 safegcd 模逆

ECDSA 签名需要 $k^{-1} \bmod n$ ，验证需要 $s^{-1} \bmod n$ 。库采用修改后的 `safegcd/divsteps`。官方实现文档给出：任意 256 位输入在 724 个 `divsteps` 内存在上界，因此秘密模逆可固定迭代次数，并用位掩码替代条件分支；对公开数据则可使用更快的 `variable-time` 版本。这体现了合理原则：秘密路径优先侧信道安全，公开验证数据路径可以使用可变时间优化。

10 Bug 修复与性能改进汇总

表 3: libsecp256k1 相关改进

版本	类型	问题/改进	密码学意义
v0.3.1	安全修复	Clang 14 条件移动产生秘密相关控制流/内存访问	防止计时与 cache 侧信道
v0.3.2	安全修复	GCC 13.1 ECDH 产生秘密相关控制流	恢复秘密标量乘常数时间性质

版本	类型	问题/改进	密码学意义
v0.4.1	性能	移除落后的 x86_64 手写域运算汇编	官方报告验证约加速 10%
v0.5.0	性能	更新签名/公钥生成点乘与预计算配置	用内存换固定基点乘速度
v0.5.1	性能	默认预计算表 22 KiB 增至 86 KiB	减少在线点加
v0.6.0/0.7.1	纵深防御	更稳健清理栈上秘密并扩大清理范围	缩短私钥材料残留时间

11 对 Bitcoin 使用密码算法的启示

1. 摘要属于协议，不只属于曲线库。Bitcoin 必须由共识规定的 sighash 序列化产生摘要，不能让网络对手直接指定 msghash32。
2. 域分离和上下文绑定不可省略。应把交易版本、输入索引、prevout 金额、Script、输出和 sighash 类型绑定到摘要，防止签名被跨上下文解释。
3. 私钥路径和公开验证路径可以不同。签名、公钥生成必须常数时间；验证只处理公开量，可安全使用 variable-time 算法提升吞吐。
4. 编译器是密码实现可信边界的一部分。升级编译器后必须重新运行 constant-time 检查、测试向量和基准。
5. 性能优化必须保持代数等价和边界证明。GLV 分解、wNAF、预计算和 safegcd 均需要明确不变量、范围界和交叉测试，不能只依据微基准合入。

12 实验限制

本实验 Python 实现使用仿射坐标和变量时间 double-and-add，其目的仅是验证公式，不能用于真实密钥。实验环境自带 CMake 3.16，低于当前仓库要求的 3.22，且未安装 Autoconf，因此没有在本机重新生成官方 C 基准数据；报告中的 10% 和 20% 数字均明确引用官方 changelog，而不是伪装成本机测量。源代码审计和摘要伪造实验不受这一构建限制影响。

13 结论

实验在不知道私钥的情况下构造了有效的 (e', r', s') ，并从代数和程序两方面证明其通过 ECDSA 验证；同时证明该签名不能验证预先指定消息的 SHA-256。这说明漏洞并不在 ECDSA 方程，而在应用是否把摘要可信地绑定到消息和业务上下文。

对官方 libsecp256k1 的检查进一步表明，高质量 Bitcoin 密码实现同时依赖：严格 API 语义、low-S 规范化、秘密路径常数时间、编译器级侧信道测试、wNAF/GLV/联合标量乘、固定基预计算以及有界 safegcd 模逆。安全与性能不是彼此独立的目标，而是在公开量与秘密量、时间与空间、通用接口与协议绑定之间进行可证明的工程权衡。

A 复现命令

```
cd 实验二
python3 ecdsa_digest_forgery.py | tee forgery_result.txt

# 记录审计的仓库提交
git -C secp256k1 rev-parse HEAD

# 查看 API 警告、安全修复与性能记录
sed -n '590,635p' secp256k1/include/secp256k1.h
sed -n '1,210p' secp256k1/CHANGELOG.md
sed -n '35,65p' secp256k1/README.md
```

B 参考资料

1. Bitcoin Core, *libsecp256k1 official repository*, <https://github.com/bitcoin-core/secp256k1>。
2. libsecp256k1, *ECDSA verification API documentation*, <https://github.com/bitcoin-core/secp256k1/blob/master/include/secp256k1.h>。
3. libsecp256k1, *Changelog*, <https://github.com/bitcoin-core/secp256k1/blob/master/CHANGELOG.md>。
4. libsecp256k1, *safegcd implementation explained*, https://github.com/bitcoin-core/secp256k1/blob/master/doc/safegcd_implementation.md。
5. T. Pornin, *Why can a signature be created without the private key when the hash is provided?*, <https://bitcoin.stackexchange.com/a/81116/35586>。

6. D. J. Bernstein and B.-Y. Yang, *Fast constant-time gcd computation and modular inversion*, <https://gcd.cr.yp.to/papers.html#safegcd>.